

Deklaratív programozás 3. előadás

Kabódi László¹ Marussy Kristóf²
BME Számítástudományi és Információelméleti Tanszék
Mesterséges Intelligencia és Rendszertervezés Tanszék

¹`kabodi.laszlo@vik.bme.hu`

²`marussy@mit.bme.hu`

2026. ősz

I. rész

Problémamegoldási technikák

- 1 Problémamegoldási technikák
- 2 For-jelölés (for-comprehension)
- 3 Műveletek, BIFek, típusvizsgálat/konverzió, mintaillesztés
- 4 Hasznos segédeszköz: dilyzer

Tartalom

1 Problémamegoldási technikák

- FPE-3 – Problémamegoldási technikák: csúszóablak használata
- FPE-3 – Problémamegoldási technikák: kihagy-bevesz rekurzió

Csúszóablakos technika: maximális összegű folytonos részlisták

Egy lista különféle szempontok szerint kiválasztott folytonos részlistáit csúszóablakos módszerrel állíthatjuk elő, egymásba ágyazott ismétlésekkel. Mivel a funkcionális nyelvekben nincs ciklus, az ismétlést rekurzióval valósítjuk meg, a részeredményeket egy, esetleg több akkumulátorban gyűjtjük. Mivel a gyűjtéshez plusz paraméter(ek)re van szükség, általában segédfüggvényeket is definiálunk.

A lista maximális összegű folytonos részlistáinak előállítására többféle megoldást is bemutat a harmadik előadás segédanyaga. Az egyes megoldások futási idejét is megmérjük a `benchee`-vel.

Egy példa:

- Bemenet (egészlista): `[1, 2, 3, 4, -10, 4, 3, 2, 1]`
- Eredmény (egy pár, első eleme a részlisták összege, második eleme ezen maximális összegű részlisták listája):
`{10, [[1, 2, 3, 4], [1, 2, 3, 4, -10, 4, 3, 2, 1], [4, 3, 2, 1]]}`

A harmadik előadás segédanyaga a honlapról letölthető:

- `dp26a-fp3ea-reszlistak-kihagy_bevesz_rek.livemd`

Tartalom

- 1 Problémamegoldási technikák
 - FPE-3 – Problémamegoldási technikák: csúszóablak használata
 - FPE-3 – Problémamegoldási technikák: kihagy-bevesz rekurzió

Kihagy-bevesz rekurzió: elemek kombinációja, kétfelé válogatása

A harmadik előadás segédanyagában két példa is van a kihagy-bevesz (*inclusion-exclusion*) rekurzióra. Az első, `komb/1` egy lista elemeinek összes kombinációját adja eredményül, a második, `eloszt/1` pedig megmutatja, hogy listaelemeket hányféleképpen lehet úgy kétfelé válogatni, hogy a részösszegek különbségének abszolút értéke a lehető legkisebb legyen. Példák a függvényhívásokra és az eredményekre:

- ❶ `komb([1,2,3]) == [[], [1], [2], [2,1], [3], [3,1], [3,2], [3,2,1]]`
- ❷ `eloszt([28, 7, 11, 8, 9, 7, 27]) == % {[9, 11, 28] => 48}`
- ❸ `eloszt([4,1,2,5,3]) == % {[4, 2, 1] => 7, [4, 3] => 7, [5, 2] => 7}`

Az `eloszt/1` eredménye egy szótár (map), melynek kulcsai az összegfeltételt kielégítő részlisták, elemei pedig az összegfeltételt kielégítő kisebbik összeg. A második példában tehát a fennmaradó `[7,7,8,27]` elemekből áll az eredményben nem szereplő másik rész, melynek összege 49.

A harmadik előadás segédanyaga, mint már írtuk, a honlapról letölthető:

- [dp26a-fp3ea-reszlistak-kihagy_bevesz_rek.livemd](#)

Dinamikus programozás: olvasnivalók, feladatok gyakorlásra

- Az *Algoritmuselmélet (VISZAA08)* tantárgy dinamikus programozásról szóló és más prezentációi: <https://www.cs.bme.hu/algel/>
- Horváth Gyula (ELTE) *összefoglalója és feladatgyűjteménye* a dinamikus programozásról: <https://people.inf.elte.hu/szlavi/VersenyFeladatok/DinaProg/dinprog.pdf>
- A Közép-európai Informatikai Diákolimpia (Central-European Olympiad in Informatics, CEOI) *versenyfeladatai*:
<http://ceoi.inf.elte.hu/tasks-archive/>
- A Nemzetközi Informatikai Diákolimpia (International Olympiad in Informatics, IOI) *versenyfeladatai*: <https://ioi.contest.codeforces.com/> (regisztráció szükséges)
- *DSA Tutorial – Learn Data Structures and Algorithms*:
<https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/> (sok példával és gyakorló feladattal), többek között a *dinamikus programozásról*: <https://www.geeksforgeeks.org/competitive-programming/dynamic-programming/>

II. rész

For-jelölés (for-comprehension)

- 1 Problémamegoldási technikák
- 2 **For-jelölés (for-comprehension)**
- 3 Műveletek, BIFek, típusvizsgálat/konverzió, mintaillesztés
- 4 Hasznos segédeszköz: dialyzer

Tartalom

2 For-jelölés (for-comprehension)

- FPE-3 – For-jelölés (for-komprehenzió, for-comprehension)

For-jelölés 1

Gyűjtemények (kollekciók) kezelésére használjuk a for-jelölést, vagy az angol elnevezést teljesen átvéve: a for-komprehenziót). Most vegyünk sorra mindent, amit a for-jelölésről tudni kell (a szögletes zárójelek jelentése itt: *opcionális*).

For-jelölés 1

Gyűjtemények (kollekciók) kezelésére használjuk a for-jelölést, vagy az angol elnevezést teljesen átvéve: a for-komprehenziót). Most vegyünk sorra mindent, amit a for-jelölésről tudni kell (a szögletes zárójelek jelentése itt: *opcionális*).

A for-jelölés: `for q1[, q2, ..., qn][, into: coll], do: exp`, ahol

- a q_i
 - 1 `pattern <- list` alakú *generátor*, vagy
 - 2 *predikátum* (igazságérték-eredményű függvény, feltétel);
- legalább egy q_i -nak *generátornak* kell lennie;
- a `pattern` mintának illeszkednie kell a `list` lista kiválasztandó elemeire, és ki kell elégítenie az adott `pattern <- list` generátortól jobbra álló összes q_i predikátumot;

For-jelölés 1

Gyűjtemények (kollekciók) kezelésére használjuk a for-jelölést, vagy az angol elnevezést teljesen átvéve: a for-komprehenziót). Most vegyünk sorra mindent, amit a for-jelölésről tudni kell (a szögletes zárójelek jelentése itt: *opcionális*).

A for-jelölés: `for q1[, q2, ..., qn][, into: coll], do: exp`, ahol

- a q_i
 - 1 `pattern <- list` alakú *generátor*, vagy
 - 2 *predikátum* (igazságérték-eredményű függvény, feltétel);
- legalább egy q_i -nak *generátornak* kell lennie;
- a `pattern` mintának illeszkednie kell a `list` lista kiválasztandó elemeire, és ki kell elégítenie az adott `pattern <- list` generátortól jobbra álló összes q_i predikátumot;
- az `exp` tetszőleges, a `pattern` mintától függő vagy nem függő kifejezés;

For-jelölés 1

Gyűjtemények (kollekciók) kezelésére használjuk a for-jelölést, vagy az angol elnevezést teljesen átvéve: a for-komprehenziót). Most vegyünk sorra mindent, amit a for-jelölésről tudni kell (a szögletes zárójelek jelentése itt: *opcionális*).

A for-jelölés: `for q1[, q2, ..., qn][, into: coll], do: exp`, ahol

- a q_i
 - 1 `pattern <- list` alakú *generátor*, vagy
 - 2 *predikátum* (igazságérték-eredményű függvény, feltétel);
- legalább egy q_i -nak *generátornak* kell lennie;
- a `pattern` mintának illeszkednie kell a `list` lista kiválasztandó elemeire, és ki kell elégítenie az adott `pattern <- list` generátortól jobbra álló összes q_i predikátumot;
- az `exp` tetszőleges, a `pattern` mintától függő vagy nem függő kifejezés;
- a generátorban a minta előállítására lista helyett más felsorolható kollekciót, leggyakrabban tartomány-típusú értéket is megadhatunk.

(Folytatás a következő dián.)

For-jelölés 2

A for-jelölés (folyt.): `for q1[, q2, ..., qn][, into: coll], do: exp`, ahol

- az opcionális `into: opció` után álló `coll`-al megadhatjuk, hogy *milyen típusú* felsorolható kollekciót hozzon létre a for-jelölés, ha elhagyjuk, alapértelmezés szerint lista jön létre;

For-jelölés 2

A for-jelölés (folyt.): `for q1[, q2, ..., qn][, into: coll], do: exp`, ahol

- az opcionális `into:` opció után álló `coll`-al megadhatjuk, hogy *milyen típusú* felsorolható kollekciót hozzon létre a for-jelölés, ha elhagyjuk, alapértelmezés szerint lista jön létre;
- `coll`-ként üres kollekciót kell megadni: `""` (sztring), `%{}` (szótár), `[]` (lista), `<<>>` (bináris);

For-jelölés 2

A for-jelölés (folyt.): `for q1[, q2, ..., qn][, into: coll], do: exp`, ahol

- az opcionális `into:` opció után álló `coll`-al megadhatjuk, hogy *milyen típusú* felsorolható kollekciót hozzon létre a for-jelölés, ha elhagyjuk, alapértelmezés szerint lista jön létre;
- `coll`-ként üres kollekciót kell megadni: `" "` (sztring), `%{}` (szótár), `[]` (lista), `<<>>` (bináris);
- a for-jelölésben definiált változók lokálisak;
- a for-jelölés *értéke* az összes olyan `expj` kifejezés kollekciója, amelyre a **mintaillesztés sikerült** és a **predikátumok teljesültek**;
- a for-jelölés eddig bemutatott változata tehát egy vagy több kollekció elemein műveletek elvégzésére és/vagy bizonyos elemek szűrésére használható (vö. `Enum.map/2`, `Enum.filter/2`).

For-jelölés 2

A for-jelölés (folyt.): `for q1[, q2, ..., qn][, into: coll], do: exp`, ahol

- az opcionális `into: opció` után álló `coll`-al megadhatjuk, hogy *milyen típusú* felsorolható kollekciót hozzon létre a for-jelölés, ha elhagyjuk, alapértelmezés szerint lista jön létre;
- `coll`-ként üres kollekciót kell megadni: `"` (sztring), `%{}` (szótár), `[]` (lista), `<<>>` (bináris);
- a for-jelölésben definiált változók lokálisak;
- a for-jelölés *értéke* az összes olyan `expj` kifejezés kollekciója, amelyre a **mintaillesztés sikerült** és a **predikátumok teljesültek**;
- a for-jelölés eddig bemutatott változata tehát egy vagy több kollekció elemein műveletek elvégzésére és/vagy bizonyos elemek szűrésére használható (vö. `Enum.map/2`, `Enum.filter/2`).
- Összefoglaló a for-jelölésről: <https://www.mitchellhanberg.com/the-comprehensive-guide-to-elixirs-for-comprehension/>
- A komprehenzió ma már sokféle programozási nyelvben megtalálható, lásd: https://en.wikipedia.org/wiki/List_comprehension

For-jelölés 3

A for-jelölés `uniq`: opcióval:

`for q1[, q2, ..., qn][, into: coll], uniq: true|false, do: exp, ahol`

- a `uniq: true` opció hatására az előállított gyűjteménybe csak egymástól különböző értékek kerülnek be;
- csak lista, sztring és bináris esetén van értelme használni, hiszen a szótárban a kulcsok sohasem ismétlődhetnek.

For-jelölés 3

A for-jelölés `uniq`: opcióval:

`for q1[, q2, ..., qn][, into: coll], uniq: true|false, do: exp, ahol`

- a `uniq: true` opció hatására az előállított gyűjteménybe csak egymástól különböző értékek kerülnek be;
- csak lista, sztring és bináris esetén van értelme használni, hiszen a szótárban a kulcsok sohasem ismétlődhetnek.

A for-jelölés `reduce`: opcióval:

`for q1[, q2, ..., qn], reduce: acc0 do acc -> fun(pat, acc) end, ahol`

- a `reduce: opcióval` a for-jelölés nem az `Enum.map/2`-et, hanem az `Enum.reduce/3`-t váltja ki,
- az `acc0` az eredményt gyűjtő akkumulátor kezdőértéke,
- a `do ... end` között egy névtelen függvényt kell megadni (az `fn` és a hozzá tartozó `end` nélkül!), amelynek egyetlen argumentuma az `acc` akkumulátor (a neve bármi lehet), a törzsében pedig egy olyan kétargumentumú függvényt vagy operátort kell használni, amelynek két argumentuma közül az egyik a generátorban használt `pat` minta, a másik pedig ugyancsak az `acc` akkumulátor (az argumentumok sorrendjének hatása lehet az eredményre!).

For-jelölés: kis példák 1

```
iex> for x <- 1..6 // 2, do: x      # { x | x ∈ {1,3,5} }  
[1, 3, 5]
```

For-jelölés: kis példák 1

```
iex> for x <- 1..6 // 2, do: x      # { x | x ∈ {1,3,5} }
```

```
[1, 3, 5]
```

```
iex> for x <- [1,2,3], do: 2*x+1    # { 2 · x + 1 | x ∈ {1,2,3} }
```

```
[3, 5, 7]
```

For-jelölés: kis példák 1

```
iex> for x <- 1..6 // 2, do: x      # { x | x ∈ {1,3,5} }
```

```
[1, 3, 5]
```

```
iex> for x <- [1,2,3], do: 2*x+1    # { 2 · x + 1 | x ∈ {1,2,3} }
```

```
[3, 5, 7]
```

```
iex> for x <- 1..9, rem(x, 2) == 0, x > 2, do: 2*x
```

```
[8, 12, 16]
```

For-jelölés: kis példák 1

```
iex> for x <- 1..6 // 2, do: x      # { x | x ∈ {1,3,5} }  
[1, 3, 5]  
iex> for x <- [1,2,3], do: 2*x+1    # { 2 · x + 1 | x ∈ {1,2,3} }  
[3, 5, 7]  
iex> for x <- 1..9, rem(x, 2) == 0, x > 2, do: 2*x  
[8, 12, 16]  
iex> for {k,v} <- [egy: 1, két: 2, há: 3], into: %{}, do: {k,v}  
%{egy: 1, há: 3, két: 2}  
iex> for {k,v} <- %{egy: 1, két: 2, há: 3}, into: [], do: {k,v}  
[egy: 1, há: 3, két: 2]  
iex> for c <- [?c, ?s, ?ó, ?k, ?a], into: "", do: <<c>>  
<<99, 115, 243, 107, 97>>  
iex> for c <- [?c, ?s, ?o, ?k, ?a], into: "", do: <<c>>  
"csoka"
```

For-jelölés: kis példák 1

```
iex> for x <- 1..6 // 2, do: x      # { x | x ∈ {1,3,5} }
[1, 3, 5]
iex> for x <- [1,2,3], do: 2*x+1    # { 2·x+1 | x ∈ {1,2,3} }
[3, 5, 7]
iex> for x <- 1..9, rem(x, 2) == 0, x > 2, do: 2*x
[8, 12, 16]
iex> for {k,v} <- [egy: 1, két: 2, há: 3], into: %{}, do: {k,v}
%{egy: 1, há: 3, két: 2}
iex> for {k,v} <- %{egy: 1, két: 2, há: 3}, into: [], do: {k,v}
[egy: 1, há: 3, két: 2]
iex> for c <- [?c, ?s, ?ó, ?k, ?a], into: "", do: <<c>>
<<99, 115, 243, 107, 97>>
iex> for c <- [?c, ?s, ?o, ?k, ?a], into: "", do: <<c>>
"csoka"
iex> for x <- 0..-2 // -2, y <- 1..x, do: {x,y}
[{0, 1}, {0, 0}, {-2, 1}, {-2, 0}, {-2, -1}, {-2, -2}]
iex> for x <- 0..-2 // -2, y <- 1..x, xy = {x,y}, do: xy
[{0, 1}, {0, 0}, {-2, 1}, {-2, 0}, {-2, -1}, {-2, -2}]
iex> for x <- 2..4, rem(x,3) != 0, y <- 1..3, x > y, do: {x,y}
[{2, 1}, {4, 1}, {4, 2}, {4, 3}]
```

For-jelölés: kis példák 2

```
iex> for i <- 1..3, j <- 2..1// -1, do: {i,j} # keresztszorzatok listája  
[{1, 2}, {1, 1}, {2, 2}, {2, 1}, {3, 2}, {3, 1}]
```

For-jelölés: kis példák 2

```
iex> for i <- 1..3, j <- 2..1// -1, do: {i,j} # keresztszorzatok listája  
[{1, 2}, {1, 1}, {2, 2}, {2, 1}, {3, 2}, {3, 1}]  
iex> for i <- 1..3, do: (for j <- 2..1// -1, do: {i,j}) # listák listája  
[[{1, 2}, {1, 1}], [{2, 2}, {2, 1}], [{3, 2}, {3, 1}]]
```

For-jelölés: kis példák 2

```
iex> for i <- 1..3, j <- 2..1//-1, do: {i,j} # keresztszorzatok listája  
[{1, 2}, {1, 1}, {2, 2}, {2, 1}, {3, 2}, {3, 1}]  
iex> for i <- 1..3, do: (for j <- 2..1//-1, do: {i,j}) # listák listája  
[[{1, 2}, {1, 1}], [{2, 2}, {2, 1}], [{3, 2}, {3, 1}]]  
iex> for i <- 1..3, j <- 2..1//-1, into: %{}, do: {i,j} # ismétlődő kulcs felülír!  
%{1 => 1, 2 => 1, 3 => 1}
```

For-jelölés: kis példák 2

```
iex> for i <- 1..3, j <- 2..1//-1, do: {i,j} # keresztaszorzatok listája
[{1, 2}, {1, 1}, {2, 2}, {2, 1}, {3, 2}, {3, 1}]
iex> for i <- 1..3, do: (for j <- 2..1//-1, do: {i,j}) # listák listája
[[{1, 2}, {1, 1}], [{2, 2}, {2, 1}], [{3, 2}, {3, 1}]]
iex> for i <- 1..3, j <- 2..1//-1, into: %{}, do: {i,j} # ismétlődő kulcs felülír!
%{1 => 1, 2 => 1, 3 => 1}
iex> for i <- 1..3, j <- 2..1//-1, into: %{}, do: {i+j*4,j} # így a kulcsok egyediek
%{5 => 1, 6 => 1, 7 => 1, 9 => 2, 10 => 2, 11 => 2}
```

For-jelölés: kis példák 2

```
iex> for i <- 1..3, j <- 2..1//-1, do: {i,j} # keresztorzatok listája
[{1, 2}, {1, 1}, {2, 2}, {2, 1}, {3, 2}, {3, 1}]
iex> for i <- 1..3, do: (for j <- 2..1//-1, do: {i,j}) # listák listája
[[{1, 2}, {1, 1}], [{2, 2}, {2, 1}], [{3, 2}, {3, 1}]]
iex> for i <- 1..3, j <- 2..1//-1, into: %{}, do: {i,j} # ismétlődő kulcs felülír!
%{1 => 1, 2 => 1, 3 => 1}
iex> for i <- 1..3, j <- 2..1//-1, into: %{}, do: {i+j*4,j} # így a kulcsok egyediek
%{5 => 1, 6 => 1, 7 => 1, 9 => 2, 10 => 2, 11 => 2}
iex> for _i <- 1..5, into: "", do: "1" # többszörözve
"11111"
iex> for _i <- 1..5, into: "", uniq: true, do: "1" # azonosak csak egyszer
"1"
```

For-jelölés: kis példák 2

```

iex> for i <- 1..3, j <- 2..1//-1, do: {i,j} # keresztorzatok listája
[{1, 2}, {1, 1}, {2, 2}, {2, 1}, {3, 2}, {3, 1}]
iex> for i <- 1..3, do: (for j <- 2..1//-1, do: {i,j}) # listák listája
[[{1, 2}, {1, 1}], [{2, 2}, {2, 1}], [{3, 2}, {3, 1}]]
iex> for i <- 1..3, j <- 2..1//-1, into: %{}, do: {i,j} # ismétlődő kulcs felülír!
%{1 => 1, 2 => 1, 3 => 1}
iex> for i <- 1..3, j <- 2..1//-1, into: %{}, do: {i+j*4,j} # így a kulcsok egyediek
%{5 => 1, 6 => 1, 7 => 1, 9 => 2, 10 => 2, 11 => 2}
iex> for _i <- 1..5, into: "", do: "1" # többszörözve
"11111"
iex> for _i <- 1..5, into: "", uniq: true, do: "1" # azonosak csak egyszer
"1"
iex> for _i <- 1..5, into: <<>>, do: <<1::size 1>> # 5 bit (0b11111), 1 byte
<<31::size(5)>>
iex> for _i <- 1..5, into: <<>>, uniq: true, do: <<1::size 1>> # 1 bit (0b1), 1 byte
<<1::size(1)>>
iex> for _i <- 1..5, into: <<>>, do: <<1::size 2>> # 10 bit (0b01010101, 0b01), 2 byte
<<85, 1::size(2)>>
iex> for _i <- 1..5, into: <<>>, uniq: true, do: <<1::size 2>> # 2 bit (0b01), 1 byte
<<1::size(2)>>

```

III. rész

Műveletek, BIFek, típusvizsgálat/konverzió, mintaillesztés

- 1 Problémamegoldási technikák
- 2 For-jelölés (for-comprehension)
- 3 Műveletek, BIFek, típusvizsgálat/konverzió, mintaillesztés**
- 4 Hasznos segédeszköz: dilyzer

Tartalom

3

Műveletek, BIFek, típusvizsgálat/konverzió, mintaillesztés

- FPE-3 – Alapműveletek, beépített függvények
- FPE-3 – Mintaillesztés

Aritmetikai és bitműveletek

- Aritmetikai műveletek (Kernel modul)
 - Előjel: +, - (precedencia: 1)
 - Multiplikatív műveletek: *, /, div, rem (precedencia: 2)
 - Additív műveletek: +, - (precedencia: 3)
- Bitműveletek (Bitwise modul)
 - bnot vagy ~~~, band vagy &&& (precedencia: 2)
 - bor vagy |||, bxor, bsl vagy <<<, bsr vagy >>> (precedencia: 3)

Aritmetikai és bitműveletek

- Aritmetikai műveletek (Kernel modul)
 - Előjel: +, - (precedencia: 1)
 - Multiplikatív műveletek: *, /, div, rem (precedencia: 2)
 - Additív műveletek: +, - (precedencia: 3)
- Bitműveletek (Bitwise modul)
 - bnot vagy ~~~, band vagy &&& (precedencia: 2)
 - bor vagy |||, bxor, bsl vagy <<<, bsr vagy >>> (precedencia: 3)

Megjegyzések

- +, -, * és / egész és lebegőpontos operandusokra is alkalmazhatók
- +, - és * eredménye egész, ha mindkét operandusuk egész, egyébként lebegőpontos
- / eredménye mindig lebegőpontos
- div és rem prefix helyzetűek, eredményük egész
- div, rem és a bitműveletek operandusai csak egészek lehetnek
- ~~~, &&&, |||, <<<, >>> infix, a többi bitművelet prefix helyzetű
- A bitműveleteket engedélyezni kell: `use Bitwise` vagy
`use Bitwise[, (only_operators | skip_operators): true]`

Összehasonlító műveletek (relációk)

- Egy reláció (összehasonlítás) eredménye a `true` vagy `false` atom
- Termek összehasonlítási sorrendje (vö. típusok):
`number < atom < reference < function < port < pid < tuple < list < binary`
- Kisebb, kisebb-egyenlő, nagyobb-egyenlő, nagyobb: `<`, `<=`, `>=`, `>`

Összehasonlító műveletek (relációk)

- Egy reláció (összehasonlítás) eredménye a `true` vagy `false` atom
- Termek összehasonlítási sorrendje (vö. típusok):
`number < atom < reference < function < port < pid < tuple < list < binary`
- Kisebb, kisebb-egyenlő, nagyobb-egyenlő, nagyobb: `<`, `<=`, `>=`, `>`
- *Érték szerinti* egyenlőség (integer és float lehet egyenlő): `==`, `!=`
- *Szigorú* egyenlőség (integer és float nem lehet egyenlő): `===`, `!==`
- Példák: `5.0 == 5 == true`, `5.0 === 5 == false`

Összehasonlító műveletek (relációk)

- Egy reláció (összehasonlítás) eredménye a `true` vagy `false` atom
- Termek összehasonlítási sorrendje (vö. típusok):
`number < atom < reference < function < port < pid < tuple < list < binary`
- Kisebb, kisebb-egyenlő, nagyobb-egyenlő, nagyobb: `<`, `<=`, `>=`, `>`
- *Érték szerinti* egyenlőség (integer és float lehet egyenlő): `==`, `!=`
- *Szigorú* egyenlőség (integer és float nem lehet egyenlő): `===`, `!==`
- Példák: `5.0 == 5 == true`, `5.0 === 5 == false`

Elrettentő példák:

`10.1 - 9.9 == 0.2` $\leadsto$ `false`

`(10.1 - 9.9) * 10` $\leadsto$

`1.9999999999999999`

`0.0000000000000001 + 1 == 1` $\leadsto$ `false`

`0.0000000000000001 + 1 == 1` $\leadsto$ `true`



Lebegőpontos értékek összehasonlítása helyett vizsgáljuk a különbségüket a `<=` vagy `>=` relációval (ϵ -nál kisebb-e a különbségük?)

Logikai műveletek

- Prefix helyzetű operátor: `not` és `!`; `not` operandusa csak `boolean`, `!`-é tetszőleges típusú kifejezés lehet
- Infix helyzetű operátorok: `and` és `&&`, `or` és `||`

Logikai műveletek

- Prefix helyzetű operátor: `not` és `!`; `not` operandusa csak `boolean`, `!`-é tetszőleges típusú kifejezés lehet
- Infix helyzetű operátorok: `and` és `&&`, `or` és `||`
 - `not`, `and` és `or` használható örökifejezésben, `!`, `&&` és `||` nem.
 - `and` és `or` első operandusa csak `boolean`, `&&` és `||` első operandusa tetszőleges típusú kifejezés lehet
 - A `false` és `nil` értékű kifejezéseket kivéve *minden más érték* `true`-nak számít.
 - Második operandusuk tetszőleges típusú kifejezés lehet
 - Eredményük típusa a két operandus típusának uniója
 - Lusta kiértékelésű, ún. *short-circuit* műveletek: ha az első operandus kiértékelése eldönti eredményt, a másodikra nem kerül sor

Logikai műveletek

- Prefix helyzetű operátor: `not` és `!`; `not` operandusa csak `boolean`, `!`-é tetszőleges típusú kifejezés lehet
- Infix helyzetű operátorok: `and` és `&&`, `or` és `||`
 - `not`, `and` és `or` használható örökifejezésben, `!`, `&&` és `||` nem.
 - `and` és `or` első operandusa csak `boolean`, `&&` és `||` első operandusa tetszőleges típusú kifejezés lehet
 - A `false` és `nil` értékű kifejezéseket kivéve *minden más érték* `true`-nak számít.
 - Második operandusuk tetszőleges típusú kifejezés lehet
 - Eredményük típusa a két operandus típusának uniója
 - Lusta kiértékelésű, ún. *short-circuit* műveletek: ha az első operandus kiértékelése eldönti eredményt, a másodikra nem kerül sor
- Példák:

```
iex> !:atom && div(3,0) === 2
false
iex> :atom && div(3,0) === 2
** ...bad argument in ...: div(3, 0)
iex> :atom and rem(3,2) === 1
** ...expected a boolean on left-side of "and"
```

```
iex> true and rem(3,2)
1
iex> false and rem(3,2)
false
iex> nil && rem(3,2)
nil
```

Beépített függvények (Built-In Functions, BIFs)

- A BEAM-be beépített, rendszerint C-ben írt függvények
- Többségük az *erts* Erlang-könyvtár `erlang` moduljának része
- Elixir-specifikációjuk az Elixir Kernel moduljában található
- A csak az Erlang `erlang` moduljában definiált BIF-ek az `:erlang` modulnévvel hívhatók

¹Megjegyzés: $0 \leq \text{index} \leq \text{tuple_size}(\text{tuple})-1$

Beépített függvények (Built-In Functions, BIFs)

- A BEAM-be beépített, rendszerint C-ben írt függvények
- Többségük az *erts* Erlang-könyvtár *erlang* moduljának része
- Elixir-specifikációjuk az Elixir Kernel moduljában található
- A csak az Erlang *erlang* moduljában definiált BIF-ek az `:erlang` modulnévvel hívhatók
- Az alaptípusokon alkalmazható leggyakoribb BIF-ek:
 - Számok: `abs(num)`, `trunc(num)`, `ceil(num)`, `floor(num)`, `round(num)`, `:erlang.float(num)`
 - `:erlang.float` helyett 1-gyel oszthatjuk az egész számot, pl. `5/1`
 - Sztring, bináris: `bit_size(string)`, `byte_size(string)`
 - Szótár: `map_size(map)`
 - Ennes: `tuple_size(tuple)`, `elem(tuple, index)`, `put_elem(tuple, index, value)`¹
 - Lista: `length(list)`, `hd(list)`, `tl(list)`
- Az operátorok is BIF-ek a Kernel-ben, pl. `Kernel.*(3,4)`

¹ Megjegyzés: $0 \leq \text{index} \leq \text{tuple_size}(\text{tuple}) - 1$

Egyéb alapfüggvények (típusvizsgálat és típuskonverzió)

- Típusvizsgálat (BIF-ek a Kernelben)

- `is_integer(term)`, `is_float(term)`, `is_number(term)`,
- `is_atom(term)`, `is_boolean(term)`, `is_nil(term)`,
- `is_binary(term)`, `is_bitstring(term)`,
- `is_tuple(term)`, `is_list(term)`, `is_map(term)`
- `is_function(term)`, `is_function(term, arity)`

Egyéb alapfüggvények (típusvizsgálat és típuskonverzió)

- Típusvizsgálat (BIF-ek a Kernelben)

- `is_integer(term)`, `is_float(term)`, `is_number(term)`,
- `is_atom(term)`, `is_boolean(term)`, `is_nil(term)`,
- `is_binary(term)`, `is_bitstring(term)`,
- `is_tuple(term)`, `is_list(term)`, `is_map(term)`
- `is_function(term)`, `is_function(term, arity)`

- Típuskonverzió (az egyes típusokhoz tartozó modulokban)

- Atom: `to_charlist(atom)`, `to_string(atom)`,
- Float: `to_charlist(float)`, `to_string(float)`,
- Integer: `to_charlist(integer)`, `to_string(integer)`,
- List: `to_atom(list)`, `to_charlist(list)`, `to_float(list)`,
`to_integer(list)`, `to_integer(list, base)`, `to_string(list)`,
`to_tuple(list)`
- String: `to_atom(string)`, `to_charlist(string)`, `to_float(string)`,
`to_integer(string)`, `to_integer(string, base)`,
- Tuple: `to_list(tuple)`,
- Map: `to_list(map)`,

Tartalom

- 3 Műveletek, BIFek, típusvizsgálat/konverzió, mintaillesztés
 - FPE-3 – Alapműveletek, beépített függvények
 - FPE-3 – Mintaillesztés

Mintakifejezés, minta, mintaillesztés (pattern matching)

- Mintakifejezés, röviden minta: termhez hasonló olyan kifejezés, amelyben nincs függvénykifejezés, de lehet benne szabad változó

Mintakifejezés, minta, mintaillesztés (pattern matching)

- Mintakifejezés, röviden minta: termhez hasonló olyan kifejezés, amelyben nincs függvénykifejezés, de lehet benne szabad változó
- Egy szabad változó mindenre illeszkedik, és lehet rá hivatkozni
- Az aláhúzásjel () és a vele kezdődő változónév mindenre illeszkedik; az előbbire nem lehet, az utóbbira nem szokás hivatkozni
- Egy mintában ugyanaz a változó többször is előfordulhat, ha mindenütt azonos értékre kell illeszkednie

Mintakifejezés, minta, mintaillesztés (pattern matching)

- Mintakifejezés, röviden minta: termhez hasonló olyan kifejezés, amelyben nincs függvénykifejezés, de lehet benne szabad változó
- Egy szabad változó mindenre illeszkedik, és lehet rá hivatkozni
- Az aláhúzásjel (_) és a vele kezdődő változónév mindenre illeszkedik; az előbbire nem lehet, az utóbbira nem szokás hivatkozni
- Egy mintában ugyanaz a változó többször is előfordulhat, ha mindenütt azonos értékre kell illeszkednie
- A mintaillesztés műveleti jele az =, bal oldalán a mintával, jobb oldalán egy tömör kifejezéssel (a mintaillesztés egyirányú)
- A mintaillesztés a minta nem fixált (vö. ^ operátor) változóit értékhez köti
- A kötés **nem** értékadás!
- Függvényhíváskor az aktuális paramétereket *illesztjük* a formális paraméterekre

Mintakifejezés, minta, mintaillesztés (pattern matching)

- Mintakifejezés, röviden minta: termhez hasonló olyan kifejezés, amelyben nincs függvénykifejezés, de lehet benne szabad változó
- Egy szabad változó mindenre illeszkedik, és lehet rá hivatkozni
- Az aláhúzásjel () és a vele kezdődő változónév mindenre illeszkedik; az előbbire nem lehet, az utóbbira nem szokás hivatkozni
- Egy mintában ugyanaz a változó többször is előfordulhat, ha mindenütt azonos értékre kell illeszkednie
- A mintaillesztés műveleti jele az =, bal oldalán a mintával, jobb oldalán egy tömör kifejezéssel (a mintaillesztés egyirányú)
- A mintaillesztés a minta nem fixált (vö. ^ operátor) változóit értékhez köti
- A kötés **nem** értékadás!
- Függvényhíváskor az aktuális paramétereket *illesztjük* a formális paraméterekre
- Ha egy változót értékhez kötünk, de nem használjuk, az Elixir figyelmeztet rá, kivéve akkor, ha a változónév _-sal kezdődik
- Figyelem: a Prologban a funkcionális nyelvekkel ellentétben *kétirányú* mintaillesztés van, *egyesítés* a neve

Példák mintaillesztésre 1

```
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]  
** (CompileError) ... & is not allowed in matches  
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]
** (CompileError) ... & is not allowed in matches
iex> [x, f] = [5, &+/2]
[5, &:erlang.+/2]
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]
** (CompileError) ... & is not allowed in matches
iex> [x, f] = [5, &+/2]
[5, &:erlang.+/2]
iex> [x, f] = [5, f]
[5, &:erlang.+/2]
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]
** (CompileError) ... & is not allowed in matches
iex> [x, f] = [5, &+/2]
[5, &:erlang.+/2]
iex> [x, f] = [5, f]
[5, &:erlang.+/2]
iex> a = fn(x) -> x+1 end
#Function<44.40011524/1 in :erl_eval.expr/5>
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]
** (CompileError) ... & is not allowed in matches
iex> [x, f] = [5, &+/2]
[5, &:erlang.+/2]
iex> [x, f] = [5, f]
[5, &:erlang.+/2]
iex> a = fn(x) -> x+1 end
#Function<44.40011524/1 in :erl_eval.expr/5>
iex> {a, b} = {fn(x) -> x+1 end, 23}
#Function<44.40011524/1 in :erl_eval.expr/5>, 23
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]
** (CompileError) ... & is not allowed in matches
iex> [x, f] = [5, &+/2]
[5, &:erlang.+/2]
iex> [x, f] = [5, f]
[5, &:erlang.+/2]
iex> a = fn(x) -> x+1 end
#Function<44.40011524/1 in :erl_eval.expr/5>
iex> {a, b} = {fn(x) -> x+1 end, 23}
#Function<44.40011524/1 in :erl_eval.expr/5>, 23
iex> fn(x) -> x+1 end = a
** (CompileError) ... fn is not allowed in matches
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]
** (CompileError) ... & is not allowed in matches
iex> [x, f] = [5, &+/2]
[5, &:erlang.+/2]
iex> [x, f] = [5, f]
[5, &:erlang.+/2]
iex> a = fn(x) -> x+1 end
#Function<44.40011524/1 in :erl_eval.expr/5>
iex> {a, b} = {fn(x) -> x+1 end, 23}
#Function<44.40011524/1 in :erl_eval.expr/5>, 23
iex> fn(x) -> x+1 end = a
** (CompileError) ... fn is not allowed in matches
iex> 3 = szabad
** (CompileError) iex:505: undefined function szabad/0
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]
** (CompileError) ... & is not allowed in matches
iex> [x, f] = [5, &+/2]
[5, &:erlang.+/2]
iex> [x, f] = [5, f]
[5, &:erlang.+/2]
iex> a = fn(x) -> x+1 end
#Function<44.40011524/1 in :erl_eval.expr/5>
iex> {a, b} = {fn(x) -> x+1 end, 23}
#Function<44.40011524/1 in :erl_eval.expr/5>, 23
iex> fn(x) -> x+1 end = a
** (CompileError) ... fn is not allowed in matches
iex> 3 = szabad
** (CompileError) iex:505: undefined function szabad/0
iex> [z | zs] = [0,1,2,3]
[0, 1, 2, 3]
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]
** (CompileError) ... & is not allowed in matches
iex> [x, f] = [5, &+/2]
[5, &:erlang.+/2]
iex> [x, f] = [5, f]
[5, &:erlang.+/2]
iex> a = fn(x) -> x+1 end
#Function<44.40011524/1 in :erl_eval.expr/5>
iex> {a, b} = {fn(x) -> x+1 end, 23}
#Function<44.40011524/1 in :erl_eval.expr/5>, 23
iex> fn(x) -> x+1 end = a
** (CompileError) ... fn is not allowed in matches
iex> 3 = szabad
** (CompileError) iex:505: undefined function szabad/0
iex> [z | zs] = [0,1,2,3]
[0, 1, 2, 3]
iex> [z1 | [z2 | [z3 | [z4 | zs]]]] = [0,1,2,3]
[0, 1, 2, 3]
iex>
```

Példák mintaillesztésre 1

```
iex> [x, &+/2] = [5, &+/2]
** (CompileError) ... & is not allowed in matches
iex> [x, f] = [5, &+/2]
[5, &:erlang.+/2]
iex> [x, f] = [5, f]
[5, &:erlang.+/2]
iex> a = fn(x) -> x+1 end
#Function<44.40011524/1 in :erl_eval.expr/5>
iex> {a, b} = {fn(x) -> x+1 end, 23}
#Function<44.40011524/1 in :erl_eval.expr/5>, 23
iex> fn(x) -> x+1 end = a
** (CompileError) ... fn is not allowed in matches
iex> 3 = szabad
** (CompileError) iex:505: undefined function szabad/0
iex> [z | zs] = [0,1,2,3]
[0, 1, 2, 3]
iex> [z1 | [z2 | [z3 | [z4 | zs]]]] = [0,1,2,3]
[0, 1, 2, 3]
iex> [z1, z2, z3 | [3]] = [0,1,2,3]
[0, 1, 2, 3]
```

Példák mintaillesztésre 2

```
iex>
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex>
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex>
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> {{a, b}, {a, b}} = {{:a, :b}, {:a, :b}}
{:a, :b}, {:a, :b}
iex>
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> {{a, b}, {a, b}} = {{:a, :b}, {:a, :b}}
{:a, :b}, {:a, :b}
iex> {{a, b, a}} = {{:a, :b, :b}}
** (MatchError) no match of right hand side value: {{:a, :b, :b}}
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> {{a, b}, {a, b}} = {{:a, :b}, {:a, :b}}
{:a, :b}, {:a, :b}
iex> {{a, b, a}} = {{:a, :b, :b}}
** (MatchError) no match of right hand side value: {{:a, :b, :b}}
iex> {a, b, _b, _} = {:a, :b, :b, :a}
{:a, :b, :b, :a}
iex>
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> {{a, b}, {a, b}} = {{:a, :b}, {:a, :b}}
{:a, :b}, {:a, :b}
iex> {{a, b, a}} = {{:a, :b, :b}}
** (MatchError) no match of right hand side value: {{:a, :b, :b}}
iex> {a, b, _b, _} = {:a, :b, :b, :a}
{:a, :b, :b, :a}
iex> {a, b}
{:a, :b}
iex>
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> {{a, b}, {a, b}} = {{:a, :b}, {:a, :b}}
{:a, :b}, {:a, :b}
iex> {{a, b, a}} = {{:a, :b, :b}}
** (MatchError) no match of right hand side value: {{:a, :b, :b}}
iex> {a, b, _b, _} = {:a, :b, :b, :a}
{:a, :b, :b, :a}
iex> {a, b}
{:a, :b}
iex> _b
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> {{a, b}, {a, b}} = {{:a, :b}, {:a, :b}}
{:a, :b}, {:a, :b}
iex> {{a, b, a}} = {{:a, :b, :b}}
** (MatchError) no match of right hand side value: {{:a, :b, :b}}
iex> {a, b, _b, _} = {:a, :b, :b, :a}
{:a, :b, :b, :a}
iex> {a, b}
{:a, :b}
iex> _b
warning: the underscored variable "_b" is used after being set...
please rename the variable to remove the underscore
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> {{a, b}, {a, b}} = {{:a, :b}, {:a, :b}}
{:a, :b}, {:a, :b}
iex> {{a, b, a}} = {{:a, :b, :b}}
** (MatchError) no match of right hand side value: {{:a, :b, :b}}
iex> {a, b, _b, _} = {:a, :b, :b, :a}
{:a, :b, :b, :a}
iex> {a, b}
{:a, :b}
iex> _b
warning: the underscored variable "_b" is used after being set...
please rename the variable to remove the underscore
:b
iex>
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> {{a, b}, {a, b}} = {{:a, :b}, {:a, :b}}
{:a, :b}, {:a, :b}
iex> {{a, b, a}} = {{:a, :b, :b}}
** (MatchError) no match of right hand side value: {{:a, :b, :b}}
iex> {a, b, _b, _} = {:a, :b, :b, :a}
{:a, :b, :b, :a}
iex> {a, b}
{:a, :b}
iex> _b
warning: the underscored variable "_b" is used after being set...
please rename the variable to remove the underscore
:b
iex> x = %{b: "barna", z: "zöld"}
%{b: "barna", z: "zöld"}
iex>
```

Példák mintaillesztésre 2

```
iex> [z1, z2, z3 | 3] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> [z1, z2 | [3]] = [0,1,2,3]
** (MatchError) no match of right hand side value: [0, 1, 2, 3]
iex> {{a, b}, {a, b}} = {{:a, :b}, {:a, :b}}
{:a, :b}, {:a, :b}
iex> {{a, b, a}} = {{:a, :b, :b}}
** (MatchError) no match of right hand side value: {{:a, :b, :b}}
iex> {a, b, _b, _} = {:a, :b, :b, :a}
{:a, :b, :b, :a}
iex> {a, b}
{:a, :b}
iex> _b
warning: the underscored variable "_b" is used after being set...
please rename the variable to remove the underscore
:b
iex> x = %{b: "barna", z: "zöld"}
%{b: "barna", z: "zöld"}
iex> %{k1: v1, k2: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
```

Példák mintaillesztésre 3

```
ix>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex> %{z: ^v1, b: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
iex>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex> %{z: ^v1, b: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
iex> %{z: v1, b: v2} = x
%{b: "barna", z: "zöld"}
iex>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex> %{z: ^v1, b: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
iex> %{z: v1, b: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"zöld", "barna"
iex>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex> %{z: ^v1, b: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
iex> %{z: v1, b: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"zöld", "barna"
iex> %{b: v} = x # részleges mintaillesztés
%{b: "barna", z: "zöld"}
iex>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex> %{z: ^v1, b: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
iex> %{z: v1, b: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"zöld", "barna"
iex> %{b: v} = x # részleges mintaillesztés
%{b: "barna", z: "zöld"}
iex> v
"barna"
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex> %{z: ^v1, b: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
iex> %{z: v1, b: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"zöld", "barna"
iex> %{b: v} = x # részleges mintaillesztés
%{b: "barna", z: "zöld"}
iex> v
"barna"
iex>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex> %{z: ^v1, b: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
iex> %{z: v1, b: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"zöld", "barna"
iex> %{b: v} = x # részleges mintaillesztés
%{b: "barna", z: "zöld"}
iex> v
"barna"
iex> ([y|ys] = yss) = [1, 2, 3] # réteges minta (layered pattern)
[1,2,3]
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex> %{z: ^v1, b: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
iex> %{z: v1, b: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"zöld", "barna"
iex> %{b: v} = x # részleges mintaillesztés
%{b: "barna", z: "zöld"}
iex> v
"barna"
iex> ([y|ys] = yss) = [1, 2, 3] # réteges minta (layered pattern)
[1,2,3]
iex>
```

Példák mintaillesztésre 3

```
iex> %{k1 => v1, k2 => v2} = x
** (CompileError) iex:3: cannot use variable k1 as map key inside a pattern...
iex> %{b: v1, z: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"barna", "zöld"
iex> %{z: ^v1, b: v2} = x
** (MatchError) no match of right hand side value: %{b: "barna", z: "zöld"}
iex> %{z: v1, b: v2} = x
%{b: "barna", z: "zöld"}
iex> {v1, v2}
"zöld", "barna"
iex> %{b: v} = x # részleges mintaillesztés
%{b: "barna", z: "zöld"}
iex> v
"barna"
iex> ([y|ys] = yss) = [1, 2, 3] # réteges minta (layered pattern)
[1,2,3]
iex> {y, ys, yys}
{[1], [2, 3], [1,2,3]}
```

IV. rész

Hasznos segédeszköz: dialyzer

- 1 Problémamegoldási technikák
- 2 For-jelölés (for-comprehension)
- 3 Műveletek, BIFek, típusvizsgálat/konverzió, mintaillesztés
- 4 **Hasznos segédeszköz: dialyzer**

Tartalom

4 Hasznos segédeszköz: dialyzer

- FPE-3 – Programok statikus analízise: dialyzer

dialyxir telepítése

```
# Run "mix help deps" to learn about dependencies.
defp deps do
  [
    {:dialyxir, "~> 1.4", only: [:dev, :test], runtime: false},
    # {:dep_from_hexpm, "~> 0.3.0"},
    # {:dep_from_git, git: "https://github.com/elixir-lang/my_dep.git", tag: ...}
  ]
end
```

Telepítsük és fordítsuk le az új modulokat!²

```
~/tmp/fp$ mix do deps.get, deps.compile
Resolving Hex dependencies...
Resolution completed in 0.051s
New:
  dialyxir 1.4.6
  erlex 0.2.7
...
Compiling ...
```

²Az új modulok az adott projekt részei lesznek, lokálisak, nem globálisak.

Szignatúravizsgálat: mix dialyzer

- A projekt forrásfájljainak a `lib` mappában kell lenniük: rakjunk be ide egy Elixir programot, pl. az egyik kisházit, rontsunk el egy-két specifikációt, és dializáljuk.
- A dializálás a `lib` mappában lévő **összes** `.ex` fájlt vizsgálja.
- Az első futtatás soká tart, mert a *dialyzer* rengeteg ún. PLT-fájlt telepít a modulokhoz tartozó típuszignatúrákkal (PLT = Persistent Lookup Table).
- A dializálás a `.beam` fájlokat elemzi, ezért ha változott valamelyik forrásfájl, az elemzés előtt fordítás készül belőle.

```
@spec sum(xs::[integer()]) :: s::[integer()]  
# Az xs számlista összege s  
def sum([x|xs]), do: x + sum(xs)  
def sum([]), do: 0  
  
~/tmp/fp$ mix dialyzer  
lib/sum.ex:2:invalid_contract  
The @spec for the function does not match the success typing ...  
Function: Sum.sum/1  
Success typing: ([number()]) -> number()  
But the spec is: (xs::[integer()]) -> s::[integer()]
```

3. gyakorlat

- Alapvetően **otthoni munka**, a gyakorlaton megbeszéljük a feladatok megoldásait
- **Gyakorlat anyaga:**
<https://dp.iit.bme.hu/dp26a/gy/dp26a-fp3gy.livemd>
- Felkészüléshez ajánlott:
https://dp.iit.bme.hu/dp26a/ea/dp26a-fp3ea-reszlistak-kihagy_bevesz_rek.livemd